

Reflective Report

ClashGuard - M A Diganta

Github Repository Link: <https://github.com/m-a-diganta/clashguard-ios>

1. Domain understanding

When I started this project I thought the hard part would be the code, but the hard part was actually understanding how casual student workers experience their week. I know this because I am one. My uni timetable lives in Canvas and my work roster lives in a completely separate system, and nothing checks the two against each other. At first I designed the app around a simple idea, just show both calendars side by side and let the student notice a clash themselves. But once I built the RosterShift model I realised the real problem is not that the information is hidden, it is that there is no time left to act on it once you do notice. That changed my thinking. I added isShortNoticeShift to RosterShift because a shift published two days before it starts is a very different situation to one published two weeks out, even if the shift itself looks identical on paper. That one property came directly from thinking about my own roster, not from a textbook definition of what a shift should contain.

2. Architecture decisions

I built three use cases, ImportRosterUseCase, DetectClashUseCase and FlagShiftForSwapUseCase, because each one protects a different rule that actually matters in real life, not just a technical step. ImportRosterUseCase protects basic data integrity, a shift cannot be added if it ends before it starts, because bad input at the start would quietly break every screen that depends on it later. DetectClashUseCase is the one I care most about, since it is the whole reason the app exists. It protects the rule that a clash should be scored by how much time is actually left, same day, day before, or just an early warning, rather than treating every overlap the same way. Getting that rule wrong in real life could mean a student ignores something urgent because the app made it look like a minor heads up.

FlagShiftForSwapUseCase protects a smaller but still real rule, a clash that has already been resolved should not be sent to the swap board again. Splitting the logic into three use cases like this also meant I could test the business rules directly, without needing a UI at all, which made me trust the app a lot more once the tests were green.

3. Human-system design

The mistake I actually ran into myself, not just imagined, was tapping the Send to swap board button more than once on the same clash. The first version of SwapBoardViewModel just appended whatever it was given, so my swap board ended up with five identical Checkout shifts stacked on top of each other. That is a genuinely confusing thing to show another casual worker, they might think five separate shifts are available when really it is one shift and four accidental taps. I fixed it by adding a guard check in the flag function that looks at the clash id before appending anything, so tapping the button again on the same clash does nothing the second time. It was a small change in code but it came from an actual mistake I made while testing the app on myself, which made it obvious that the interface needed to protect the user from their own double tap rather than trusting every tap to be intentional.

4. What I would do next

With two more weeks I would replace InMemoryRosterRepository and InMemoryDeadlineRepository with something that reads real data instead of sample data, most likely a Canvas calendar import for deadlines and a photo scan for the roster. I chose this as the next step because the architecture is already set up for it, ImportViewModel only talks to the RosterRepository and DeadlineRepository protocols, not to any specific data source, so swapping in a real implementation would not require touching the use cases, the views, or the tests at all. That is really the whole point of using repositories in the first place, and it feels like the most honest next step rather than the most exciting one, since it would finally close the gap between the demo and something a real student could use with their own roster.